

Level 2: Applied

All Librarians

Module 10

Prompt library for library work

The difference between occasional AI users and power users is a prompt library. This module teaches you to build one, systematically capturing what works so you don't start from scratch every time.

By Yulia Brusova

20 min read · Reviewed October 2026

WHAT YOU'LL BE ABLE TO DO

- 1 Build a personal prompt library with at least ten reusable prompts for your library work
- 2 Set up a Claude Project (or equivalent) with persistent context for a recurring workflow
- 3 Share prompts effectively with colleagues so they can adapt them for their own use
- 4 Explain why a prompt library reduces variability in AI output
- 5 Develop a lightweight process for adding to and improving your prompt library

The most consistent pattern among librarians who use AI effectively is not that they have better tools or more time to experiment; it is that they have stopped starting from scratch. The first time a prompt produces genuinely useful output, most people note that it worked and then move on. The next time they need a similar output, they reconstruct the prompt from memory, get a different result, and spend time adjusting. A prompt library is the professional practice of not letting that happen: systematically capturing what works, organizing it for retrieval, and building on it over time rather than repeatedly rediscovering the same effective approaches. The investment required is modest. The return, compounded across every task where you would otherwise start from scratch, is substantial.

What a prompt library is and why variability is the problem it solves

A prompt library is a curated collection of prompts and the contextual notes that make them reusable, organized for recurring professional tasks. It is not a log of every conversation you have had with an AI tool, and it is not a repository of prompts you have tried and might try again. It is a set of prompts that have been tested, have produced consistently useful output, and have been documented in a form that allows you to use them again without reconstructing them from memory.

The problem a prompt library solves is variability. AI output quality is not fixed; it varies significantly with how the prompt is constructed, what context the model has access to, and what constraints the prompt imposes. A librarian who drafts reference email responses by

typing a quick request each time will get different quality outputs on different days, depending on how much context she happens to include, how clearly she specifies the tone and length, and whether she happens to articulate the task well in the moment. A librarian who runs the same email inquiry through a tested, context rich prompt template gets consistent output that she edits rather than rewrites. Such a difference (between inconsistent outputs that require substantial revision and consistent outputs that require light editing) is the practical consequence of working with and without a prompt library.

The analogy that resonates most with librarians is the reference interview framework. A reference librarian who has developed a reliable set of clarifying questions for common inquiry types does not reconstruct those questions from scratch with each patron; she has internalized an approach that reliably produces the information she needs to help. A prompt library externalizes the same logic: the "clarifying questions" are embedded in the prompt structure, and the librarian applies a tested approach rather than improvising each time. Furthermore, just as a reference interview framework improves with use, retaining questions that consistently produce useful responses and revising those that produce confusion, a prompt library improves as the librarian learns which prompt structures produce reliable results for her specific professional context.

What makes any prompt work

The prompts in this module are starting points, not formulas - they should be adapted to the specific task and context each time. What makes any prompt work is what makes any professional communication work: adequate context, clear expectations, and an honest account of what is actually needed. Dakan and Feller identify a consistent pattern across effective AI prompts: they specify not just what to produce, but who the output is for, what constraints apply, and how the AI should approach the task (Dakan & Feller, *AI Fluency: Framework & Foundations*, Anthropic Academy, 2025). The most common reason a prompt underperforms is that it named the desired output but not the purpose or audience. For example, the difference between asking for "a bibliography on food insecurity" and asking for "five annotated sources on food insecurity appropriate for a first-generation college student writing a persuasive essay" is the difference between a usable result and an editing project.

Additionally, when a prompt is not producing what is needed, the most efficient move is often to ask the AI directly to help improve it - describing the actual goal and asking for a better way to phrase the request. In practice this sounds like: *"I'm trying to create a research guide for students who have never used a database, but I'm not getting what I need. Can you help*

me ask this better?" Such an approach treats the AI as a thinking partner in the communication task itself, which is often faster than iterating on the prompt alone.

What belongs in a library prompt library: categories and examples

The most useful starting point for building a prompt library is not to create prompts for every possible task, but to identify the five or six tasks you perform most frequently where AI assistance has already proven useful, and to build tested prompts for those tasks first. Such a focused beginning produces a small library that is actually used, rather than a comprehensive library that is consulted occasionally. The library grows naturally as the librarian encounters additional tasks where a saved prompt would have saved time.

For most academic librarians, the highest value categories fall into four areas. The first is patron communication: reference email responses by inquiry type, FAQ content for specific services, and subject guide descriptions for specific disciplines and patron populations. A tested prompt for reference email responses that specifies institution type, patron population, preferred tone, length constraint, and what to include or avoid will produce a useful draft for the vast majority of inquiries with only the patron's question swapped in. For example: "You are a library associate at a community college serving a high proportion of first generation students. A student has sent the following reference inquiry: [paste question]. Write a 150-word email response in a warm, encouraging tone that directly addresses the question, points the student to the most appropriate starting resource, and offers to follow up. Do not use library jargon." Such a prompt, once tested and confirmed to produce output that requires only light editing, is worth saving with that exact phrasing.

The second category is instruction: lesson plan drafts, learning objective sets, active learning activity ideas, and assessment questions. A tested lesson plan prompt that specifies session length, patron population, course context, the assignment being supported, and the specific learning outcome to prioritize will produce a starting structure for almost any single shot instruction session. The third category is administrative writing: annual report sections, grant narrative segments, policy draft frameworks, and meeting agendas, all tasks that share the characteristic of being time consuming, structurally familiar, and amenable to AI drafting with light human review. The fourth category is research support: search strategy development for specific disciplines, document summarization with specified output format, and synthesis across multiple sources for a stated purpose.

A practical rule for deciding what to add to the library: if you used an AI prompt for a task and thought "I should save that," save it within the same session. If you wait until later, the exact prompt wording is typically gone, and you have only a general memory of what worked. In order to make saving effortless, maintain the library in a format you can access immediately (a pinned document, a saved note, a dedicated tab) rather than in a system that requires multiple navigation steps before you can add an entry.

Anatomy of a reusable prompt: variables, context, and constraints

The structural difference between a single use prompt and a reusable prompt is the presence of variables: clearly marked placeholders for the elements that change from task to task, and the explicit specification of context and constraints that remain constant across uses. A single use prompt is optimized for a specific task in a specific moment; a reusable prompt is optimized for a class of tasks and documents the parameters that make it work.

The standard structure of a reusable library prompt has four components. First, a role or context specification that tells the model who it is and what professional setting applies: "You are a reference librarian at a midsize public university library serving undergraduate and graduate students." Such a specification does not need to change between uses; it is the constant context that orients every output. Second, a task description with bracketed variables for the elements that change: "A [patron type] has sent the following inquiry: [paste inquiry]. Draft a [length]-word response in a [tone] tone." Third, a constraint set specifying what the output should include or avoid: "Include a recommended starting database, a brief explanation of why it is appropriate for this inquiry, and an offer to follow up. Do not use library jargon or assume prior database experience." Fourth, an output format specification when the format matters: "Format the response as a complete email, beginning with a greeting and ending with your name and title."

The bracketed variables ([patron type], [inquiry], [length], [tone]) are where the prompt is adapted for each use. Everything else stays constant. For example, the reference email prompt described above might be used for an undergraduate research inquiry, a graduate thesis consultation inquiry, and a faculty course reserve inquiry in the same week, with only the patron type and the pasted inquiry changing between uses. Such consistency in the non variable components is what produces consistent output quality: the model receives the same professional context, the same constraint set, and the same format specification each time, and the variability in output reflects only the genuine variability in the task itself rather than variability in how well the prompt happened to be constructed on a given day.

A practical note on prompt length: more specific prompts produce more consistent output, but there is a point at which adding constraints produces diminishing returns. The prompts in a well functioning library are typically longer than the prompts a librarian would construct spontaneously (more context, more explicit constraints, more format guidance) because they incorporate everything learned from testing about what produces reliable output. In order to find that specification level for a given task, the most efficient approach is to start with a reasonable prompt, test it on three or four representative examples of the task, note where the output required the most significant editing, and revise the prompt to address those failure modes. Such iterative refinement (test, identify failures, revise) is what distinguishes a tested prompt from a drafted one.

Claude Projects and persistent context: setting up your professional workspace

A prompt library stores what to ask; persistent context stores who is asking and why. The two work together. A librarian who has built a reference email prompt library still benefits from a persistent context setup that means she never has to include her institution type, patron population, and professional role in every prompt, because the model already knows that information from the project context. Such a combination, namely a library of tested task specific prompts used within a persistent professional context, is the configuration that produces the highest quality outputs with the least per session setup.

Claude Projects is the current implementation of persistent context in Claude. A project is a defined workspace with its own conversation history, custom instructions that apply to every conversation within the project, and uploaded documents the model can reference throughout. For example, a reference and instruction librarian might set up a project with custom instructions that include: "I am a reference and instruction librarian at a community college with a large first generation and returning adult student population. My institution has limited database subscriptions: primarily JSTOR, Academic Search Complete, and ProQuest. My patron facing communication should always use plain language, avoid library jargon, and assume students may be anxious about library research." Such instructions, set once and applied to every conversation in the project, eliminate the need to reestablish professional context at the start of every session.

Uploaded documents within a project extend the persistent context to include institution specific reference materials. A librarian who uploads her library's current database list, its hours and services FAQ, its information literacy outcomes document, and two or three of her strongest existing lesson plans creates a knowledge base the model draws on in every

conversation, without the librarian needing to paste that content into each prompt. For example, when drafting a subject guide for a nursing course, the model can reference the uploaded database list to confirm what databases are actually available at the institution rather than suggesting databases the library does not subscribe to. Such grounding in institutional reality is a significant improvement over models generating generic outputs based on training data alone.

ChatGPT's Custom Instructions and Custom GPTs serve a similar function, with different architectural characteristics. Custom Instructions in ChatGPT apply persistent context across all conversations, making them appropriate for general professional context. Custom GPTs create a distinct configured assistant with specific instructions, uploaded knowledge, and defined capabilities, making them more appropriate for a specific recurring workflow than for general professional context. For shared team applications, Custom GPTs can be shared within an organization's ChatGPT Business or Enterprise plan, which is a relevant consideration for libraries where prompt sharing and collaborative workflow development are priorities. The practical guidance for most library professionals is to use whichever persistent context mechanism is available in the tool they use most consistently; the architecture differs, but the professional value of not restating your professional context in every session is the same regardless of platform.

Sharing prompts across your library: multiplying individual effort

A prompt that works for one librarian will work for her colleagues, with adjustments for their specific context. The institutional value of a prompt library is not fully realized until the prompts are shared, because an individually maintained library represents one person's accumulated knowledge, while a shared library represents the team's accumulated knowledge, and the latter is substantially more valuable than any individual contribution to it.

The simplest sharing mechanism is a shared document such as a Google Doc or Notion page organized by task category, with prompts presented in a standardized format that includes the prompt text, a single sentence description of the output it produces, the task it is designed for, any notes on how to adjust it for different contexts, and the AI tool on which it was developed and tested. Such a format matters because prompts shared without context are rarely adopted. A colleague who sees a block of prompt text with no explanation of when to use it or what it produces has no basis for evaluating whether it is relevant to her work. A colleague who sees the same prompt with a note that reads "Use this for drafting reference email responses to undergraduate research inquiries; it produces a 150-word draft in an

accessible tone that typically needs only minor edits" can immediately evaluate whether it applies to her workflow and how to adapt it for her institution's context.

The LibGuide format is particularly well suited to institutional prompt sharing because it is familiar infrastructure that library staff already know how to navigate, and because it allows prompts to be organized by task type with tabs or boxes in a way that supports browsing rather than requiring keyword search. A staff only LibGuide page for library AI prompts (visible to colleagues but not to patrons) provides a natural home for a shared prompt library that does not require setting up new technology. For example, a LibGuide organized with tabs for Patron Communication, Instruction, Administrative, and Research Support prompts, with each prompt presented in the standardized format described above, is immediately usable by colleagues who have never contributed to its development.

The social dimension of prompt sharing deserves direct attention. Prompts are professional knowledge: a well crafted reference email prompt represents accumulated expertise about what makes patron communication effective in a specific institutional context. In order to make sharing feel like contribution rather than extraction, the acknowledgment practices that accompany prompt sharing matter: noting who developed a prompt, inviting colleagues to submit prompts they have developed, and treating the shared library as a team resource rather than an anonymous collection. Additionally, prompts should be shared with explicit permission to adapt; the goal is not that every librarian uses the exact same text, but that tested prompts serve as starting points that colleagues modify for their own context rather than building from scratch. Such a norm, to adapt rather than copy wholesale, produces a library that improves across the team rather than converging on a single approved text.

Maintaining a living prompt library: dating, testing, and pruning

A prompt library that is not maintained degrades. AI models update on vendor schedules that are not always announced clearly, and a prompt that produced reliable output on one model version may produce different output after a model update; sometimes better, sometimes worse, sometimes just different in ways that require the library to be updated. Tasks evolve as the librarian's workflow changes, institutional priorities shift, and the tools available to patrons change. A prompt written for a database the library no longer subscribes to, or for an instruction approach the librarian has moved away from, is not merely unused; it is actively misleading if a colleague consults the shared library and acts on it.

The minimum maintenance practice is dating every prompt when it is added to the library. A date entry does not require ongoing effort; it simply records when the prompt was last confirmed to work as described. Such a date allows anyone consulting the library to make an

informed judgment about whether a prompt is likely to still be current; a prompt tested eight months ago in a field that has seen significant AI tool changes requires more skepticism than one tested last month. A "last tested" field, updated each time a prompt is used and confirmed to produce the expected output, is the single most useful maintenance addition beyond the initial date.

Annual review is the appropriate cadence for pruning. Once per year, or when a significant AI model update occurs, the library should be reviewed for prompts that are no longer relevant, prompts that reference tools or resources that have changed, and prompts that have been superseded by more effective versions developed since the original entry. Such a review is not a project; it is a few hours of evaluation, deletion, and updating that prevents the library from becoming cluttered with obsolete entries that reduce its usability. For example, a prompt library that has been accumulating entries for two years without review may contain three different versions of the reference email prompt, each representing a different stage of the librarian's thinking, with no indication of which is current. A pruned library contains one, the best current version, clearly dated.

The final maintenance practice is version noting for significant revisions. When a prompt is substantially changed (not minor wording adjustments but a structural revision that changes what the prompt produces), it is useful to note what changed and why. For example: "Revised October 2025: added constraint on response length after observing that earlier version produced outputs patron recipients found too long." Such a note is not required for every small adjustment, but it is useful for changes that reflect a meaningful learning about what the prompt needs to include. Furthermore, for shared libraries, version notes allow colleagues to understand why the prompt they had been using has changed, which prevents the confusion of having a prompt produce different output than expected without understanding why the standard version was updated. A prompt library maintained with dating, testing, and periodic pruning remains a genuinely useful professional resource rather than an archaeological record of past experiments.

PRACTITIONER NOTE

A common resistance to building a formal prompt library is the perception of overhead: one more thing to maintain. The moment that typically changes the calculation is a concrete cost - spending twenty minutes trying to reconstruct a prompt that worked well three weeks earlier, unable to reproduce the output quality because the exact phrasing is gone. The time spent trying to reconstruct it exceeds the time it would have taken to save it properly the first time. That is a specific, measurable loss, not a

hypothetical future efficiency. Librarians who begin saving prompts consistently after such a moment often find that six months later the library has forty or more entries covering most recurring tasks. The ones used weekly save time that is immediately noticeable. The ones used monthly save time that adds up across a semester. The ones saved but not yet used cost nothing except thirty seconds to capture. The ratio is favorable.

KEY TAKEAWAYS

- A prompt library solves the variability problem: the same well constructed prompt produces consistent output that requires light editing, while reconstructed prompts produce variable output that requires substantial revision.
- The highest value starting categories for a library prompt library are patron communication, instruction, administrative writing, and research support; begin with the five or six tasks you perform most frequently where AI has already proven useful.
- A reusable prompt has four structural components: a constant role or context specification, a task description with bracketed variables for elements that change, a constraint set specifying what to include or avoid, and an output format specification.
- Claude Projects and equivalent persistent context features (ChatGPT Custom Instructions and Custom GPTs) eliminate the need to reestablish professional context in every session; prompts used within a persistent project context produce better grounded outputs with less per session setup.
- Prompts shared without context are rarely adopted; share prompts with a single sentence description of what they produce, the task they are designed for, and explicit permission to adapt for different institutional contexts.
- A prompt library that is not maintained degrades; the minimum maintenance practice is dating every prompt when added, reviewing and pruning annually, and noting significant revisions so colleagues understand why a standard prompt has changed.

References

APA 7TH EDITION

1. Dakan, R., & Feller, A. (2025). *AI fluency: Framework & foundations* [Online course]. Anthropic Academy. <https://anthropic.skilljar.com/ai-fluency-framework-foundations>

ACRL AI Competencies covered

Use & Application

Sub-competencies: 4.3, 4.1 · ACRL AI Competencies (2025)